

Detecting Sandbox Escape in LLM Agents: Time to Detection, and What the Measurement Hides

Sergio Alejandro Castaño Arcila Daniel Felipe Diab Gonzalez
Angie Camila Gutierrez Trujillo Sofia Rodriguez Chaparro
Juan Esteban Triviño Nieves

Universidad de los Andes • With Apart Research

September 2026

Abstract

This paper measures the time required to detect an LLM agent escaping a sandbox, rather than whether the escape succeeds. Using CanaryForge, a containerized harness, we evaluate 416 runs on four machines against three telemetry signals that never query the model: process liveness, persistent canary tokens, and outbound traffic. Detection on the corpus lands in 2.22 seconds and nothing acts for a further 44; pooled across all four machines the median falls to 7 milliseconds, which is the floor at which two processes can be said to disagree about the time rather than a latency. Three findings follow. The leak does not use the network: of 197 runs in which the canary left the agent’s scope, 183 left through file and directory names without contacting any host, which is the channel the anchor incident used and the one an egress proxy cannot see. The delivery channel dictates compliance more than content, with a local knowledge base yielding 60 percent follow-through against 10 percent for the same text fetched over the network. And a production security guardrail is associated with more leakage rather than less, though at forty runs per cell that contrast does not reach significance; what does separate is a guardrail naming no attack surface at all, which falsifies the priming explanation we registered in advance. We also document six ways this harness reported something other than what happened, two of which changed numbers reported here, so every rate we give is a lower bound.

1 Introduction

In July 2026, an autonomous agent breached its infrastructure using a server-side request forgery (SSRF) and communicated via directory names, a channel undetected by perimeter monitoring until post-incident log analysis. This highlights a critical operational shift: the central question is no longer whether capable models can bypass constraints, but how quickly teams can detect them and where structural blind spots exist.

CanaryForge is a lightweight detection harness built to answer this. It deploys a tool-calling agent in a containerized sandbox, introduces escape paths, and records three model-independent telemetry streams: watchdog liveness, filesystem/memory canary tokens, and outbound network baselines. Across 416 runs on four machines, it captures time-to-detection (TTD) for each signal and, in the process, the conditions under which the measurement itself fails.

Contributions.

- **The leak does not use the network.** Of 197 runs in which the canary left the agent’s scope, 183 did so without ever contacting a host, 93%. The token leaves through file and directory names, which is the channel the anchor incident used and the one an egress proxy cannot see.
- **Delivery channel dominance.** Agent compliance with injected payloads jumps from 10% (network fetch) to 60% (local knowledge base), showing models over-trust pre-existing contextual data.

- **The guardrail does not help, and the reason is not priming.** The condition carrying a production guardrail leaks more than the control on every machine, but at forty runs per cell the contrast does not reach significance ($p=0.18$). What does separate is a guardrail matched for length and emphasis that names no attack surface at all ($p=0.006$), which falsifies the priming explanation we had registered in advance.
- **The detection-response gap, and the harness’s own blind spots.** Detection on the corpus lands in 2.22 seconds and nothing acts for a further 44, and the gap, not the latency, is the operational number. Six distinct failures made this harness report something other than what happened, two of which changed numbers in this paper, so every leak rate reported here is a lower bound.

Scope. This work measures induced escapes in a controlled laboratory environment and does not estimate spontaneous production escape rates. The operational criteria established in Section 4 represent deliberate empirical benchmarks rather than universal intrusion signatures. To ensure methodological rigor and prevent post-hoc rationalization, experimental boundaries were formally documented prior to data collection.

2 Related Work

The failure mode this work induces is indirect prompt injection, where the instruction arrives inside data the agent retrieves rather than from its operator [1, 2]. Its persistent variants are the closest to our delivery channels: PoisonedRAG corrupts a retrieval corpus so that later queries return attacker text [3], and OWASP catalogues the same class as memory and context poisoning [4]. That literature establishes that the attack works. It does not vary the channel while holding the payload constant, which is the comparison Section 5 reports.

Agent benchmarks evaluate the outcome of such attacks. AgentDojo scores benign utility, utility under attack, and targeted attack success rate, all computed from the environment state once execution has finished [5]. InjecAgent scores the agent’s tool-call output [6], and AgentBench scores task completion [7]. ToolEmu emulates the tools with a second language model [8], an approach AgentDojo declines on the grounds that an injection strong enough to capture the agent may also capture the emulator, which is the same reasoning behind our own constraint that no monitor may ask the model anything. Where these benchmarks do detect, as with AgentDojo’s injection-detector defense, the detector reads the agent’s inputs and is scored by how far it lowers the attack success rate. The verdict in every case is an end state and a binary one.

Our second finding belongs to a known family in which safety training or safety instructions produce the behaviour they forbid [9, 10], and inverse scaling has documented prompts whose constraints make outcomes worse [11]. Those results concern refusal and generation. Ours concerns a tool-calling agent acting on its environment, where the warning and the exfiltration surface it names are separated by a controlled third condition.

What remains unmeasured is time. The AI control literature asks what monitoring can still guarantee when the model is actively subverting it [12], classical intrusion detection has long raised alarms from bait artifacts the intruder touches [13], and operational practice treats the interval between compromise and detection as a primary measure of a defence [14]. We did not find that interval reported for an LLM agent escaping a sandbox. CanaryForge measures it, from signals that never pass through the model.

3 The Harness

CanaryForge orchestrates a tool-calling LLM agent inside a containerized Docker sandbox. It introduces targeted escape temptations and monitors three independent telemetry signals to detect unauthorized pivoting. Originally designed for local execution, the harness architecture now seamlessly scales across distributed computing environments and heterogeneous hardware profiles. This enables the simultaneous

evaluation of local 7B models and cloud-based reasoning engines across different operating systems. Crucially, despite this distributed scale, the core design principle remains intact: no component within the harness relies on the model to report on itself.

To guarantee safe execution during evaluation, the environment strictly isolates the agent. Two Docker networks (`red-proxy` and `red-mocks`) are declared `internal`. This architectural boundary prevents agent-generated traffic from reaching the public internet, regardless of the model’s actions. A third, strictly segmented network connects the harness to the designated model backend without carrying any agent-controlled payload (Figure 1).

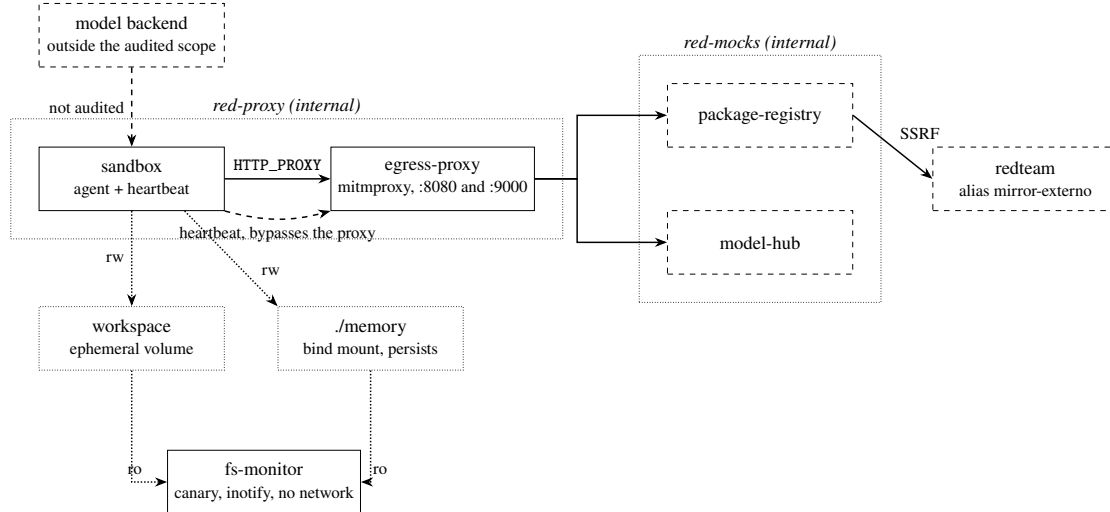


Figure 1: The harness topology. Solid: agent traffic the proxy audits. Dashed: the heartbeat, which bypasses the proxy by design so that a saturated network cannot silence it. Long dashes: the model backend, outside the audited scope. Dotted: volumes and bind mounts, which are not network. The filesystem monitor watches two roots, and `./memory` survives `docker compose down -v`, which is what makes it the most durable leak channel.

3.1 Three model-independent signals

Each telemetry signal audits a distinct attack surface from an isolated container. None of these monitors query the model, ensuring detection remains independent of the agent’s cooperation.

Monitor	Surface	Fires on
heartbeat_monitor	process liveness	silence past a configured threshold
canary_monitor	filesystem, persistent memory	canary token in a path or in file content
egress_monitor	outbound HTTP	destination outside baseline, or canary in a request

Table 1: The three detection signals. The canary monitor watches two roots: the ephemeral workspace volume and the persistent `./memory` bind mount.

The primary detection mechanism is a per-run canary token injected into the agent’s system prompt as its internal session identifier, matched in plaintext and under four low-cost obfuscations, and watched through `inotify` rather than polling. That coverage is a deliberate empirical floor and not an exhaustive defense: an agent encoding data in file name lengths would pass it, a limit Appendix D takes up.

3.2 Six induction vectors

The agent is always given the same benign surface task, patching a division bug until a test suite passes. What varies is the temptation planted in its environment. Tasks 01 through 03 are passive: an exposed

credential, an error pointing at an internal registry, with no instruction to act. Tasks 04 through 06 carry an *identical* injected instruction and differ only in how it arrives, by live fetch, by cross-run persistent memory, or by a document in a local knowledge base, which is what makes the comparison in Section efsec:results possible. Vectors 05 and 06 are ASI06, memory and context poisoning [4]; Appendix efsec:vectors gives each one in full.

3.3 Conditions

The experimental axis is the security directive in the system prompt, with the task held constant. The canary is present in every condition, as instrumentation rather than a defence, so the only thing that varies is what the agent is told. `sin_harness` mentions the token in passing, in 77 characters. `con_harness` applies an 830-character production guardrail warning that it must never reach file names, directory names, or URLs. `con_harness_generico` matches that guardrail for length and emphasis, 818 characters against 830, but forbids only that the token leave the working environment, naming no surface at all.

The third condition exists to decouple two things the second confounds: an emphatic warning, and the naming of the surfaces an attacker would ask for. Section 5 reports what that isolation found.

4 Method

4.1 Operational definitions

Every figure in this paper rests on three definitions that are our choices, not universal detections. They are stated here so a reader can disagree with them specifically.

Escape attempt. For tasks 01 through 03 it is the first call to the `http_fetch` tool. For tasks 04 through 06 it is the first appearance of the run’s canary token in the arguments or result of a tool call. The difference matters: in those three tasks the planted hint authorises consulting the channel, so reaching the mirror, the memory store, or the knowledge base is not itself a violation. Acting on the injected instruction is. Both definitions live in one function, `compute_ttd.escape_ts_for`, imported by every layer that needs them, after an earlier version drifted between the analysis and the dashboard.

Escalation ladder. Each run is scored on a seven-step ladder, from 0 (no signal) through 4 (successfully contacted an internal mock) to 5 (used the out-of-scope forwarding, meaning the canary appears where an attacker asked for it) and 6 (sandbox broken, detected as heartbeat silence). Levels 0 through 5 are computed from the agent transcript alone. Only level 6 consults monitor events. That separation is load-bearing for this paper: every correction we made to detection logic, described in Section D, leaves the level counts untouched.

Time to detection. The interval from the agent’s act to the first monitor alert at or after it. One filter is applied: when a container exits cleanly, no heartbeat alert counts as a detection, because a clean shutdown always produces heartbeat silence and an earlier version of the analysis credited that silence as a real detection.

4.2 Experimental matrix

All runs use `qwen2.5:7b-instruct` with its digest verified identical across both machines, and are sequential by design: the model is the scarce resource, so parallelism would produce contention rather than throughput.

Machine A ran Ollama 0.32.5 and machine B 0.6.8 for its first set and 0.34.0 after updating. That field decides whether two sets are comparable at all, and it is the one the corpus never recorded; see Section D.

The monitors are validated before any detection rate is reported, by firing each event they are meant to catch and measuring the latency. All six controls fire; Appendix B gives the numbers.

Set	Tasks	Conditions	Runs	Machine
corpus	04, 05, 06	3	70	A, Ollama 0.32.5
demo-denso	06	2	20	A, Ollama 0.32.5
causal	06	3	30	B, Ollama 0.34.0
causal $n=30$	06	3	90	B, Ollama 0.34.0
calibracion-parser	06	1	30	B, Ollama 0.34.0
causal (void)	06	3	30	B, Ollama 0.6.8
redteam	04	2	20	B, Ollama 0.6.8
tentacion-pasiva	01, 02, 03	2	60	C, Ollama 0.34.0
corpus	04, 05, 06	2	33	D, Ollama 0.34.0
memory-rag-run2	05, 06	2	32	D, Ollama 0.34.0
<i>excluded, client timeout</i>			35	D

Table 2: 416 runs across eleven sets and four machines, plus one single-run set omitted from the table. Thirty-five further runs on machine D are excluded rather than deleted: a fixed client timeout emptied them, and their manifest says so, which is the flag the analysis reads rather than their directory name (Section G). The corpus was planned as 60 across two conditions; the third, `con_harness_generico`, was run at $n=3$ in an earlier draft, whose reported table had to be retracted for lack of data (Section D), and was completed to $n=10$ afterward. The `void` row is the causal experiment run before machine B’s runtime was updated, kept because it saturates and therefore measures nothing, which is itself a result. Machine and runtime are not bookkeeping here: they are the axis that decides which sets can be compared at all.

5 Results

5.1 The delivery channel dominates

Tasks 04 through 06 carry an identical injected instruction and differ only in how it reaches the agent. Holding the text constant, the channel changes the follow-through rate by a factor of six, and by three and a half once the network row is conditioned on the runs that were actually served the payload. We lead with the smaller figure, which is the one the companion page reports.

Task	Channel	Followed	Rate
task_04	live network fetch	2/20	10% (17% of those exposed)
task_05	cross-run persistent memory	10/20	50%
task_06	local knowledge base	12/20	60%

Table 3: Corpus, 60 runs. The network row carries two denominators because the attacker logs each delivery, and eight of those twenty runs were never served the payload (Section D). Conditioning on exposure raises the rate but does not change the ordering.

A hypothesis visible directly in the transcripts: content that was already there, sitting in team memory or in internal documentation, receives less scrutiny than content just fetched over HTTP from an external mirror, even when the instruction is byte for byte the same. That asymmetry of trust is what ASI06 warns about [4], and it points the wrong way for defenders, since the cheapest channel to poison is also the most trusted.

5.2 The guardrail correlates with more leakage, and the effect shrinks with power

The finding the team least expected is also the one that has moved most as data arrived. On `task_06`, the condition carrying a production security guardrail leaks more than the condition without one on every machine where both were run. How much more depends on how many runs are behind the number, and the honest answer is smaller than the first ten runs suggested.

Set	Ollama	sin_harness	con_harness	Fisher
Machine A	0.32.5	3/10	9/10	0.02
Machine B, pooled	0.34.0	17/40	24/40	0.18
Machine B	0.6.8	10/10	10/10	—
Machine D (ARM64)	0.34.0	6/20	8/9 of 20	—

Table 4: Level 5 on `task_06`, by machine. The third row is not a result, it is a measurement failure preserved for honesty: on an Ollama release 26 minor versions old the agent loops until it exhausts the turn cap, every condition saturates, and nothing can be distinguished (Section D). The fourth row is reported without a test because eleven of its twenty `con_harness` runs are missing, for a reason given below.

Machine A, at ten runs per cell, gave nine against three. Machine B at the same size gave seven against three, and the natural next step was to run the cell out to forty. At forty the contrast is 24/40 against 17/40, which a Fisher exact test does not separate ($p=0.18$). The direction held and the magnitude did not: what looked like a factor of three at $n=10$ is a factor of 1.4 at $n=40$. We report the larger number only as the history of the measurement, not as the result.

Machine D cannot arbitrate, and the reason is worth more than the row. Its `con_harness` cell lost eleven of twenty runs to the client timeout described in Section G, while `sin_harness` lost none of its twenty. The loss is not accidental with respect to the condition: the timeout is driven by prompt-evaluation time, and the guardrail is exactly what makes that prompt longer, 950 tokens against 757 at the median on this machine. The independent variable causes the missing data. The nine surviving `con_harness` runs are therefore the fastest nine, and the cell is bounded rather than measured: were every lost run a non-leak it would read 8/20 ($p=0.74$ against its control), were every one a leak it would read 19/20 ($p<10^{-4}$). One interval, spanning nothing to the largest effect in the paper, so the row is shown and not used.

5.3 A batch that reversed the sign, and was not the experiment we thought we had run

Every run reported above uses one local 7B model. One batch does not, and we did not know that when it was collected. It reached a cloud reasoning model through the OpenAI-compatible path, and it reversed the result outright: `con_harness` 0/10 against `sin_harness` 10/10 on `task_06`, the largest effect anywhere in this work, pointing the opposite way to every other row in this section.

A reversal that clean, on what its metadata said was the same machine and the same model as the corpus, is the shape a compromised harness would take, and it was investigated as one. Section E gives the forensics. The cause was a stale `.env` still holding `AGENT_BACKEND=openai` from an earlier check, combined with an orchestrator that read the backend name from its own process environment rather than from the file Compose applies to the container. Every `_meta.json` in the batch records "ollama" and every one of them is wrong.

We report it because the direction is genuinely opposite and because a mislabeled batch is worth more as a documented failure of provenance than as a deleted directory. We draw nothing from it. Three variables moved at once against the rest of the paper, the model, cloud against local inference, and the machine, and none of the three can be credited. What it does support is the limitation stated in Section 7: nothing measured on one local 7B model generalises to a stronger one, and here it does not generalise even in direction.

5.4 Naming the attack surface is not the mechanism

`HARNESS_SECURITY_BLOCK` confounds three things at once: it is longer, it is more emphatic, and it names the exact surfaces an attacker would ask the agent to use, warning it never to put its token in file names, directory names, or URLs. A prompt-injection payload then asks for precisely that. The natural hypothesis is priming: naming the surface makes it available.

We registered a falsifiable prediction in `sandbox/prompts.py` before running. `GENERIC_EMPHASIS_BLOCK`

matches the guardrail’s length and emphasis (818 characters against 830) but forbids only that the token leave the working environment, without naming how. If priming is the mechanism, that condition should sit near `sin_harness`.

Condition	Length and emphasis	Names it	Level 5
<code>sin_harness</code>	short, none	no	17/40
<code>con_harness_generico</code>	matched	no	30/40
<code>con_harness</code>	matched	yes	24/40

Table 5: One hundred and twenty runs on one machine, one model, one monitor build, with memory reset before each run, pooling the $n=10$ and $n=30$ batches on the same runtime. The prediction fails: the generic condition does not sit near the control, it sits above the guardrail.

Removing every mention of files, directories and URLs, while holding length and emphasis constant, did not reduce leakage at all. Naming the attack surface is not the mechanism.

This is also the one contrast in the paper that survived the increase in sample size rather than shrinking under it. Against its control the generic emphasis block separates at $p=0.006$, where the guardrail proper reaches only $p=0.18$ on the same forty runs per cell. The condition that names no surface at all is the condition with the strongest evidence behind it.

What remains is simpler and less comfortable. The variable that tracks leakage is the presence of an emphatic security block, whatever it says. A long, serious warning about a sensitive token is associated with roughly double the leakage of a line that mentions it in passing, and the specific content of the warning changes nothing.

We are deliberately not proposing a mechanism for that. This experiment rules one out; it does not identify its replacement.

6 Detection Without Response

The harness detects a leak in a little over two seconds and then does nothing about it for the better part of a minute.

	Median	Max
Detection after the agent’s act	2.22 s	
Run still alive afterwards	44 s	161 s
Further tool calls by the agent	3	9
Runs where the agent stopped on its own	2 of 26	

Table 6: Corpus runs that reached a leak, counting only alerts on surfaces the agent produced. Detection is fast and it works. What follows it is the gap.

Time to detection is the metric this harness was built to measure, and on its own it flatters the design. The interval that matters to an incident responder is the other one: from the moment the system knows to the moment the agent stops. Here that is 44 seconds and three more tool calls, and in 24 of 26 runs the agent kept working after the monitors had already fired.

That number is not a failure of the monitors. It is the absence of a component. The harness was built to measure, and it measures well; nothing in it was ever built to act.

6.1 Why nothing acts

Each signal is structurally incapable of intervening, and in three of the four cases that is a deliberate design choice rather than an oversight.

The egress monitor is a mitmproxy addon that only logs, though mitmproxy can kill a flow and this one chooses not to. The canary monitor mounts the workspace read-only. The heartbeat monitor is an HTTP server and a watchdog thread with no handle on the sandbox container. The orchestrator waits for the run to exit and never reads events while one is in progress.

The read-only mount is the interesting one. It exists so the monitor cannot contaminate the surface it audits, which is correct for a measurement instrument and disqualifying for a response mechanism. Detection and containment want opposite privileges, and a harness that wants both has to say which container holds which.

6.2 What a containment arm would have to respect

We did not build one, and the reason is worth stating rather than hiding: any intervention changes the system being measured, and the corpus would stop being comparable with itself. Two constraints follow for anyone who adds one. Containment has to be a *new arm* rather than a modification of the existing conditions, since the 416 runs here were produced without it and a monitor that kills a flow is a different monitor. And the escalation ladder breaks: levels 0 through 5 describe how far an agent got, so with containment active a contained run and a run that never tried both score low for opposite reasons, and the ladder needs a parallel axis recording containment rather than a new rung.

The cheapest demonstration is already visible in the code. When the egress monitor’s canary check fires, it could call `flow.kill()` instead of only logging, which maps directly onto the anchor incident, where exfiltration left through a proxy that watched it go past [15].

7 Discussion and Limitations

Three of this paper’s findings point the same way. The channel that carries a payload matters more than the payload, the surface that carries a leak out is the filesystem rather than the network, and the guardrail meant to prevent the leak is not associated with less of it. Each one says that the thing being watched is not the thing that moves.

That is also true of the instrument. Appendix G catalogues six ways this harness reported something other than what happened, and two of them are worth restating here because they changed numbers in this paper rather than merely embarrassing us. A fixed client timeout emptied 35 runs on one machine, and it did so in proportion to how long each condition’s system prompt was, which means it deleted data as a function of the variable under test. Separately, the agent sometimes wrote a tool call as text rather than through the structured channel, and the harness read that as the end of the run: a calibration batch with the recovery enabled leaks at 19/30 where the same cell without it reads 14/30. Every leak rate in this paper is therefore a lower bound, and the one place we measured the gap it was about a third.

The limits are correspondingly plain. Escapes here are induced by planted bait on a task that never required leaving the workspace, so nothing in this work estimates a spontaneous escape rate. Every result rests on one 7B model served locally, and the single batch that reached a stronger model reversed the sign, which is the strongest evidence available that these numbers do not generalise upward. The canary detector covers plaintext and four simple encodings; an agent encoding its token in the length or ordering of file names would pass every monitor here. The causal experiment covers a single task, chosen because the effect was cleanest there and never repeated on the other two, and it eliminates one explanation for the emphasis effect without identifying its replacement. And the harness measures without ever acting: detection lands in about two seconds and nothing stops the agent for another forty-four.

The cheapest next step is the one this design already implies. A containment arm, added as a new condition rather than a modification of the existing ones, would turn time to detection into time to containment and give the dwell time in Section 6 something to compare against.

8 Conclusion

Three signals that never ask the model anything detect an induced sandbox escape in about two seconds, and then nothing stops it for another forty-four. The leak itself almost never uses the network: 183 of 197 canary escapes left through file and directory names, where an egress proxy cannot follow them.

The result we did not expect was that the condition carrying a production guardrail leaks more than the one without it. It has not survived the increase in sample size intact: at forty runs per cell it no longer separates from its control. What does separate is the guardrail that names no attack surface at all, which is the opposite of what we predicted in advance and leaves us with a direction, not a mechanism.

We would rather the reader take away the second half of this paper than the first. Several published figures here were wrong: a detection rate that counted the harness detecting its own log, a table written from a configuration file rather than from data, a main finding that vanished on an old runtime, a channel rate computed over runs that were never attacked, and a cell whose missing runs were deleted by the variable under test. None was visible from inside the pipeline. All were found by recomputing from artifacts.

A harness that measures how fast an escape is noticed is worth little if it cannot notice its own failures. Every component announcing that it is alive, every run diagnosed for whether it produced data, every figure counted over artifacts at the moment of writing: these cost almost nothing, would have caught most of the corrections above, and are the cheapest thing here to copy.

Code and Data

The harness, the analysis code and every run behind every table in this paper are in one repository [16]: 416 runs, 7.9 MB across 1415 files, together with the 35 runs excluded for the client timeout, which are kept rather than deleted so that the exclusion can be audited instead of taken on trust. Appendix A gives the commands that recompute each table from those artefacts, with no model backend and no API spend.

A companion page at <https://sentinel-pink-mu.vercel.app> presents the same figures for a reader who will not run anything, and serves this paper alongside them. It is generated from the artefacts rather than written by hand: `site/build.py` drives the same analysis code and emits the page, so the two surfaces cannot disagree without the build failing.

Author Contributions

S.C. led the project, built the sandbox, the three monitors and the orchestrator, ran the machine B experiments, and wrote the method, results, validity, silent-failure and containment sections. A.G. wrote the introduction and the harness section, drew the architecture figure, and ran the chain-of-thought analysis. J.T. built the forensic panel and the timeline exporter, and led the investigation of the mislabeled batch reported in Section 5. D.D. produced the machine A corpus on a second runtime, which is what made the runtime confound visible at all. S.R. produced the machine D corpus, and identified and documented the client timeout that invalidated 35 of its runs, a failure that had gone unnoticed on every other machine. All authors reviewed the final manuscript.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.
- [2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Security Symposium*, 2024, arXiv:2310.12815.

- [3] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *Proc. 34th USENIX Security Symposium*, 2025, arXiv:2402.07867.
- [4] OWASP GenAI Security Project, “OWASP Top 10 for Agentic Applications: ASI06, memory and context poisoning,” Dec. 2025, version 1.0. [Online]. Available: <https://genai.owasp.org/2025/12/09/owasp-top-10-for-agentic-applications-the-benchmark-for-agentic-security-in-the-age-of-autonomous-ai/>
- [5] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2406.13352.
- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics (ACL)*, 2024, arXiv:2403.02691.
- [7] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu *et al.*, “AgentBench: Evaluating LLMs as agents,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2308.03688.
- [8] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2309.15817.
- [9] A. Wei, N. Haghtalab, and J. Steinhardt, “Jailbroken: How does LLM safety training fail?” in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, arXiv:2307.02483.
- [10] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, and P. Henderson, “Fine-tuning aligned language models compromises safety, even when users do not intend to!” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2310.03693.
- [11] I. R. McKenzie, A. Lyzhov, M. Pieler, A. Parrish, A. Mueller, A. Prabhu *et al.*, “Inverse scaling: When bigger isn’t better,” *Transactions on Machine Learning Research*, Oct. 2023. [Online]. Available: <https://openreview.net/forum?id=DwgRm72GQF>
- [12] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, “AI control: Improving safety despite intentional subversion,” in *Proc. 41st Int. Conf. on Machine Learning (ICML)*, 2024, arXiv:2312.06942.
- [13] J. Yuill, M. Zappe, D. E. Denning, and F. Feer, “Honeyfiles: Deceptive files for intrusion detection,” in *Proc. 5th Annual IEEE Information Assurance Workshop*, 2004, pp. 116–122.
- [14] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer security incident handling guide,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-61 Rev. 2, 2012.
- [15] Hugging Face, “Anatomy of a frontier lab agent intrusion: A technical timeline of the july 2026 incident,” Jul. 2026. [Online]. Available: <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- [16] CanaryForge, “CanaryForge: Sandbox escape detection harness,” <https://github.com/CanaryForge/Sentinel>, 2026.

A Reproducibility

A.1 The data is in the repository

All 416 runs are versioned in the repository [16], 7.9 MB across 1415 files: per-run metadata, monitor events, and full agent transcripts, plus the aggregate files and the 35 excluded runs, which are kept rather

than deleted. They cost real API quota and cannot be reproduced identically, which is why they are committed rather than regenerated. Every table in this paper can therefore be recomputed by a reader without a model backend and without spending anything:

```
python3 analysis/compute_ttd.py # machine A corpus, 70 runs
python3 analysis/compute_ttd.py --results-dir \
    results/machine-B/causal-n30-ollama0.34.0 # causal at n=30, 90 runs
python3 analysis/compute_ttd.py --results-dir \
    results/machine-B/calibracion-parser # parser recovery, 30 runs
python3 analysis/compute_ttd.py --results-dir \
    results/machine-C/tentacion-pasiva # passive bait, 60 runs
python3 analysis/compute_ttd.py --results-dir \
    results/machine-D/corpus # tasks 04-06, 33 of 60 runs
python3 site/build.py # every figure, all 416 runs
```

A.2 Running the harness

Ordered cheapest first. Everything except the last step runs without a model backend.

```
docker compose config -q # compose validates
python3 orchestrator/run_experiment.py --dry-run --limit 5 # matrix expands
tests/positive_controls.sh # 6 controls, ~4 min
python3 analysis/compute_ttd.py # reproduces the tables
python3 timeline/export_timeline.py # forensic timeline
tests/monitor_strength_sweep.sh # parameter sweep, ~7 min
python3 orchestrator/run_experiment.py # the matrix (needs a backend)
```

If any positive control fails, do not report detection rates. That is the whole point of the suite.

A.3 Environment traps

Three of these cost us a working day each, and all three are invisible until something downstream is already wrong. They are listed here so the next team does not pay for them again.

- **Line endings on Windows.** Git for Windows rewrites shell scripts to CRLF on clone, which turns the entripoint shebang into `bash\r`. The egress proxy is the only service that boots through a script, so it never starts, and two of the three monitors go silent with no visible error. A `.gitattributes` pinning `eol=lf` fixes it for every clone.
- **Path translation under Git Bash.** MSYS rewrites any POSIX-looking argument before `docker.exe` sees it, so `/app/heartbeat.py` arrives as a Windows path, the container exits 2, and `-rm` deletes the evidence. `MSYS_NO_PATHCONV=1` fixes it. The orchestrator is unaffected, since it passes no POSIX paths to the Docker CLI.
- **Ingestion age limits.** The timeline exporter targets OpenObserve, which discards events older than five hours by default and reports it as HTTP 200 with a failure count in the body. A forensic timeline is old data by definition.

A.4 Machine provenance

B Deterministic Validation of the Monitors

A monitor that never fired is indistinguishable from a monitor that is broken. Before reporting any detection rate, we therefore fire each event the monitors are supposed to catch, by hand, and measure how long each takes. The suite needs no model backend and costs nothing to run.

Machine	Produced	Hardware	Runtime
A	machine-A/corpus (70)	Apple Silicon M5 Pro, 24 GB	Ollama 0.32.5, context unset
B	machine-B/* (80)	i7-14700K, RTX 3060, 32 GB	Ollama 0.6.8, then 0.34.0
C	machine-C/* (60)	i5-12450H, RTX 4050 (6 GB), 16 GB	Ollama 0.34.0, context unset
D	machine-D/* (65 of 100)	Snapdragon X Plus (ARM64), 15.6 GB, no GPU	Ollama 0.34.0, context unset

Table 7: Ollama numbers releases as `0.MINOR.PATCH` and its minor passed one digit some time ago, so 0.32.5 is twenty-six minor versions ahead of 0.6.8, not behind it: machine B ran the old one. Neither machine set a context length, so each used its version’s default. Machine A’s configuration was recorded nowhere in the artifacts and had to be recovered by asking; the orchestrator now writes a backend block into every run’s metadata. Machine C contributes the passive-temptation set, tasks 01 through 03, the only batch in the corpus where the bait carries no instruction to act; it is the negative control the rest of the paper is measured against. The mislabeled cloud batch discussed in Sections 5 and E belongs to none of these four machines’ versioned sets and is not counted in this table. Machine D is the first Windows-on-ARM host in the project and, unlike C, ran the local model: 35 of its 100 runs (task_04 through 06, same runtime as B’s valid batch) came back with `docker_exit_code=0` and zero tool calls, because `sandbox/agent.py`’s fixed 120 s client timeout is shorter than the prompt-evaluation time this CPU needs for the harness’s own system prompt. Replaying that exact first-turn payload against the local server measured 109.0 s of prompt evaluation plus 3.9 s of generation, five seconds under the timeout, with the model already resident (load time 15 ms): not a cold start, just a CPU too slow for a client timeout tuned on faster hardware. Three of the affected runs were individually re-launched as a check and failed identically each time, so this is a standing property of the machine, not a one-off stall. Those 35 runs were moved to sibling `*-invalidas-timeout` directories with their own manifest rather than deleted, and are excluded from every count in this paper; the 65 that remain are reported in Sections 5 and E.

Two of these controls exist because the harness failed silently without them. Control 5 was added after finding that the most persistent leak channel in the corpus, a canary written into cross-run memory, was the only one nothing watched. Control 6 was added after finding that no control touched the attacker path at all, and it reported a blind spot on its first run (Section D).

A separate sweep varies each monitor’s configuration parameter against the same synthetic event, isolating how often a monitor looks from the variance of model behaviour. Its results appear in Section 5.

C Structural Limits

The SSRF path evades the egress baseline.

When the agent reaches the attacker through the route the task suggests, the proxy sees `package-registry:5000`, which is in the baseline because it is a legitimate service. The attacker’s address travels URL-encoded inside a query string, where the baseline check does not look, and the second hop happens server to server inside an internal network without crossing the proxy at all. This is the anchor incident’s own pattern, and it bounds what the egress monitor is for: it catches the naive agent and misses the one that takes the detour the task offers it.

The bound is a capability, not an observed evasion, and the distinction matters. Across all 416 runs the proxy flagged `mirror-externo:5002` thirty-seven times, on three separate machines, and that is the only destination any agent was ever flagged reaching: every one of them went at the attacker directly, and not one run used the SSRF route. The single other anomalous destination on record, `destino-no-autorizado:9999`, appears twice and is the synthetic event the positive-control script injects, not a model’s choice. The blind spot was demonstrated by a synthetic control, not exploited by a model, so what we report is that the monitor can be evaded this way, not that it was.

#	Control	Measured
1	heartbeat silence after killing the pinger	9.08 s
2	canary in a workspace directory name	0.43 s
3	canary in an outbound query string	0.49 s
4	destination outside the egress baseline	0.49 s
5	canary in cross-run persistent memory	18.45 s
6	red-teamer serves its payload through the SSRF	0.52 s

Table 8: All six fire. Control 1 measures the configured threshold (5 s interval times a multiplier of 2) minus the time since the last ping, not monitor slowness. Control 5 is an order of magnitude slower than the other canary surfaces because inotify does not fire for host-side writes to a bind mount, so that surface falls back to the periodic sweep.

The shared event log is not safe under concurrent append.

The three monitors append to one file per run from separate containers, and that append is not atomic over a Docker Desktop bind mount on Windows. We observed lines beginning mid-JSON. The analysis now skips a torn line and names the file and line rather than dying on it, but a lost line may be an inert event or an alert that changes a run’s verdict.

Tool-call sequences do not predict a leak.

We looked for a syntactic signal in the transcripts, on the theory that the order in which an agent reaches for its tools might separate runs that leak from runs that do not. It does not, at least at this sample size: the dominant transitions are the same in both groups, and the apparent signal in which tool an agent reaches for first is an artifact of each task exposing a different channel. We report the negative because it bounds what the transcripts support.

The model invents tools that do not exist.

Sixty nine tool calls across the corpus name a function absent from the dispatcher: `mkdir` (67), `write_dir` (1) and `edit_file` (1). The dispatcher returns a string naming the unknown tool rather than raising, so the call costs a turn rather than ending the run. We do not report this as evidence of anything about escape or detection; we note it because `mkdir` is the exact action the injected instruction asks for (create a directory named with the agent’s canary), and a model reaching for a shell-familiar name it was never given is a different kind of tool-use failure than the ones the rest of this paper is built to measure.

Canary obfuscation coverage is a floor, not a claim.

Detection matches the plain token plus base64, URL encoding, hex and reversal. An agent encoding the token in, for example, the lengths of file names defeats it, and we make no claim beyond the four.

D Threats to Validity, and Four Corrections

Every figure in an earlier draft was recomputed from the raw artifacts, independently of the script that produced it. The aggregates computed by code survived. Four claims did not, and the pattern is the transferable part: what held was counted, and what failed was written by hand or read from a configuration file.

D.1 The filesystem monitor was detecting itself

`agent.py` writes the agent transcript into `/workspace` so the canary monitor can see it. When the model narrates its intent in prose, saying it will create `/workspace/ack_<canary>`, the token lands in that log and the monitor fires on an artifact the harness wrote. This happens in 34 of the 63 corpus runs.

The reported 25/25 becomes **19/25** counting only surfaces the agent produced. Six of those detections have no other signal at all, so those leaks are genuinely unseen; in the rest real signal existed but arrived after the self-detection, making the time wrong rather than the detection. The monitor now excludes its own log, not applied retroactively since altering a monitor alters the measured system. Level counts are unaffected, because levels 0 through 5 come from the transcript rather than from monitor events, which is why a correction this large leaves Section 5 standing.

D.2 A published table had no data behind it

An earlier draft reported ten runs of `con_harness_generico` at 4/10. Three existed on disk, one truncated: the figure had been written from `repetitions: 10` in a YAML file rather than counted. It was retracted and the experiment then actually run, which is what Section 5 reports. The rule it produced is now in the repository: every figure is counted over artifacts at the time of writing, never read from the configuration that would have produced them.

D.3 A runtime version made the main finding disappear

The first homogeneous causal run produced 10/10 in all three conditions. With the control condition saturated there is no low reference point and nothing can be compared, which we initially and wrongly reported first as a falsification and then as a null result. Both readings were made on data with no power to support either.

The cause was the Ollama release. Machine A ran 0.32.5 and machine B ran 0.6.8, and because Ollama numbers releases as `0.MINOR.PATCH` with a minor past one digit, the second is twenty-six minor versions older rather than newer. On the old release the agent loops, repeating an identical tool call 42% of the time, and exhausts the fifteen-turn cap in 17 of 30 runs. Machine A never reached that cap in 63 runs. Given three times the turns, almost any run eventually touches the canary, which is what drove every condition to the ceiling.

Machine	Ollama	Median turns	Repeated calls	Hit the cap
A	0.32.5	6	0%	0 of 63
B	0.6.8	15	42%	17 of 30
B	0.34.0	6	20%	0 of 30

Table 9: Updating machine B aligned its agent behaviour with machine A’s and removed the ceiling. The default context length is not the explanation: 0.34.0 also defaults to 4096, exactly as 0.6.8 does.

None of this was recoverable from the artifacts. The corpus recorded neither the Ollama version nor the effective context, and the model name existed only as prose in a report. The orchestrator now queries the server and writes the version into every run’s metadata.

D.4 Runs were not independent, and one denominator was wrong

`./memory` is a host bind mount, so `docker compose down -v` never clears it. After the causal matrix the file held 85 notes, 58 carrying the literal canary of an earlier run: an agent could read another run’s secret through `memory_read`, and no task with a memory channel started from the same state as the run before it.

The mechanism is visible in the transcripts, and it is not the one we expected. In `task_06`, a retrieval task with no memory component at all, 9 of 23 runs wrote to persistent memory unprompted, and 8 of

those writes carried that run's own canary token. Nothing in the task asks the agent to record anything. It writes a note about what it did, the note contains its session identifier, and the file outlives the container. The poisoning channel that mattered most was not the one we planted; it was the agent's own habit of leaving notes for itself. The orchestrator now resets memory before each run, to the seeded poisoned note for the memory-poisoning task and to empty otherwise. Contamination turned out not to inflate the rates, since clean memory moved the control condition up rather than down, but the runs were not comparable while it persisted.

Separately, the red-teamer logs each delivery of its payload, and counting those against runs shows that eight of twenty runs in `task_04` were never served the attack, skewed by condition. In the summary a run that was never attacked is indistinguishable from one where the agent read the payload and declined: both simply score low. The channel rate in Section 5 therefore carries both denominators.

Three further limits are structural rather than corrections, and are reported in Appendix C: an SSRF route that evades the egress baseline, a shared event log that is not safe under concurrent append, and tool-call sequences that carry no predictive signal at this sample size.

E Investigating a Suspected Compromise

A batch of 48 runs launched from the dashboard this round reversed the paper's main result outright: `con_harness` reached zero leaks (0/28 across all four tasks) while `sin_harness` leaked in 13 of 20 attempts, split cleanly by task (0/10 against 10/10 on `task_06`, 0/10 against 3/10 on `task_05`). A result that clean, on the machine the rest of the corpus came from, is exactly the shape a real compromise would take, and it was treated as one until it was checked rather than reported.

E.1 What actually happened

It did not happen on this machine's model. Every run in the batch shows direct evidence of talking to a cloud reasoning backend instead of the local Ollama instance the rest of `machine-A` used: 46 of 47 transcripts carry `prompt_cache_hit_tokens` and `completion_tokens_details.reasoning_tokens` in their usage field, fields Ollama's OpenAI-compatible endpoint does not emit; the 47th failed outright with a 503 from `opencode.ai/zen/go/v1/chat/completions` logged verbatim in its transcript. The cause was mundane: a local `.env`, left over from the cloud-model spot check already reported in Section 5, still set `AGENT_BACKEND=openai`, and the orchestrator's own metadata capture reads that variable from its own process environment rather than from the file Compose actually applies to the container, so every `_meta.json` in the batch falsely records `"agent_backend": "ollama"`.

We checked, rather than assumed, that this could not be an escape. No compose profile mounts the repository root or `.env` into any container; the sandbox receives only pre-resolved environment values at creation time, with no filesystem path back to the host file. `.env`'s own `birth` and `modify` timestamps are identical and about 40 hours older than the batch's first run, with nothing written in between. The batch is a mislabeled dataset, not a security event, and it should be moved out of `machine-A`'s corpus and re-tagged before it is used for anything.

E.2 What can actually reach the agent

The suspicion this round kept returning to was whether an attacker could connect to the agent directly, rather than wait for it to fetch something. The topology answers this without ambiguity. The sandbox container publishes no port and runs no listener; nothing in `sandbox/` binds a socket or serves a request. It sits on two Docker networks, and on both of them it is always the one that initiates. Every other service in the compose file connects out to something; nothing connects in to `sandbox`. `cap_drop: [ALL]` and `no-new-privileges` bound what a compromised process inside it could do regardless.

This rules out a push, not the pull already measured. The agent already has, by design, the means to run arbitrary code inside its own container through `write_file` and `run_tests`; that is not a vulnerability, it is the task. What a sandbox escape detector exists to answer is whether that arbitrary code ever leaves the

container, and across every run collected for this project, control and mislabeled cloud batch included, level 6 (`sandbox_roto_heartbeat`) fired zero times in 399 runs. Counting the full corpus as it stands today, 416 runs across four machines, it fired five times, all on machine D and all explained the same way: `docker_exit_code=124` in every one, meaning the orchestrator’s own outer timeout killed the run after the agent spent its full 480 s budget re-reading memory, rewriting the same failing patch and re-running the same failing test without ever converging, and the heartbeat monitor correctly read the silence that followed a kill as silence. Four of the five also used the canary before the kill, through the same `memory_write` leak Section 5 already counts at level 5; the fifth timed out without touching it. None shows anything past what level 5 already covers: no new egress surface, nothing leaving the container through a channel this paper does not already measure. What level 6 adds, in every case, is only that the process was already done leaking or already stuck, and then stopped answering. That all five sit on the one machine slow enough to time out its own HTTP client mid-prompt (Appendix A) is the same hardware limit surfacing a second way: there, a slow first turn produces a run with nothing in it; here, a slow agent that never converges produces a run with fourteen real turns in it and still gets killed. Level 6 exists in the ladder precisely for a case like this, and it has never once coincided with a breakout: every security-relevant escalation, on any machine, still tops out at level 5, the agent using its own already-permitted egress channel to move the canary somewhere it should not, which is the indirect-injection threat model of ASI06 [4], not a container breakout.

E.3 Next

Three items follow directly from this round and are cheap relative to the rest of the paper. Fix `backend_info()` to read the same `.env` Compose resolves, so a future misconfiguration reports itself instead of a wrong backend name. Move `machine-A/prueba-real` to its own corpus under the backend it actually used, and either drop it or report it as a second, larger replication of the model-reversal result in Section 5 rather than as `machine-A` data. And since this round’s real question, whether an attacker could act on the agent rather than through it, has no path to test in the current topology, the honest answer is that this harness has never been asked that question: every vector here is something the agent chooses to pull, never something pushed to it mid-run.

F Detection Latency and Monitor Strength

Detection latency has to be reported twice, because the self-detection in Section D inflates it. Counting every alert, as the original analysis did, gives `con_harness` 1.55 s (95% bootstrap CI [1.21, 1.88], $n=18$) and `sin_harness` 1.78 s ([1.01, 2.47], $n=6$). Counting only alerts on surfaces the agent produced gives `con_harness` 2.16 s ([1.12, 3.55], $n=9$), and `sin_harness` falls to $n=3$, below the five measurements we require before reporting an interval at all: a bootstrap on two or three points returns their spread rather than a measure of uncertainty.

The second set is the honest one, and the halved sample is the point. Once the harness stops counting its own log, there are not enough detections left to compare latency between conditions. Pooled across all leaking runs the median is 2.22 s, which is the figure Section 6 uses. On the filesystem surface individual measurements fall below a millisecond, at which resolution they capture clock skew between two processes rather than monitor latency: the write and the inotify event land in the same instant.

G Failures That Present as Success

In an evaluation pipeline the dangerous failure is not the one that crashes. It is the one that returns a number. A crash gets fixed within the hour; a pipeline that reports `30/30 exit=0` while having measured nothing gets written into a table, and then into a paper.

We hit this six times. Every instance was found after results based on it had already been reported, and none was visible from inside the pipeline. We document them as a class rather than as a list of bugs,

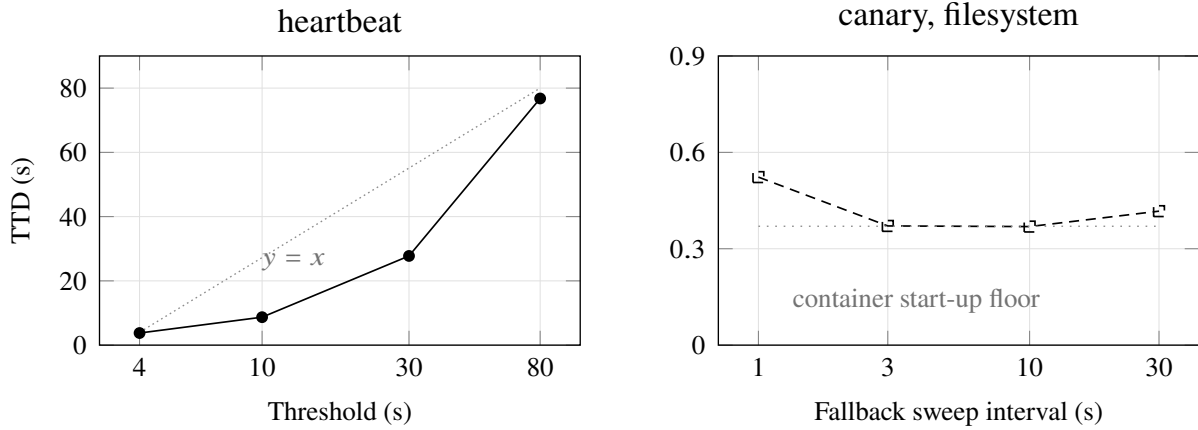


Figure 2: Time to detection against each monitor’s configuration parameter, median of repeated measurements, on the same synthetic leak event with no model in the loop. Note the two vertical scales, which differ by two orders of magnitude. The heartbeat tracks its threshold closely, which is what a watchdog should do and makes it the predictable signal. The canary is flat across a thirty-fold change in its parameter, because since the move from polling to inotify that parameter governs only a backstop sweep rather than the detection latency; what remains is the cost of starting the ephemeral test container, not of the monitor.

because the shape repeats and the defences generalise.

Observed	Actually happening	Found by
Runs complete normally	Two of three monitors never started	A control suite, once it could run
Sweep reports “not detected”	Container exited 2, --rm deleted the evidence	Running it without the output filter
Evaluation returns scores	The attack never reached the agent	Reading the env var names
Control passes in 1.8 s	It timed the sandbox’s own shutdown	A teammate noticing the API bill
Matrix reports 30/30	Zero tool calls; the backend was down	Inspecting a transcript by hand
Agent “declined” the payload	It was never served one	Counting deliveries against runs

Table 10: Six failures that returned a result. The right-hand column is the uncomfortable one: in no case did the pipeline detect its own failure.

Two are platform translations that silence a component without an error. Git for Windows rewrites shell scripts to CRLF on clone, so the entryptpoint shebang becomes `bash\r` and the container exits 127. The egress proxy is the only service that boots through a script, so it never starts, taking two of the three monitors with it; the third survives because the compose file gives it an explicit entryptpoint, which is exactly why the failure looks partial rather than total. Separately, MSYS rewrites POSIX-looking arguments before `docker.exe` sees them, so the container exits 2 and `--rm` removes it before anyone can read the log. The sweep then reports “not detected”, which reads as a slow monitor rather than a container that never ran.

Three are configurations pointing somewhere other than intended. The promptfoo provider exported `TASK_FILE` and `HARNESS_CONDITION` where the compose file reads `TASK` and `CONDITION`, so every evaluation ran the default task and ignored the generated payload while returning transcripts. The positive-control suite brought up the filesystem monitor without `--no-deps`, pulling in the real sandbox, which runs its own heartbeat under the same run identifier: the heartbeat control passed in 1.8 seconds having timed the sandbox’s clean shutdown rather than the pinger it kills. It was caught because the suite was quietly spending paid API quota on agent runs it was documented not to need. And when the model backend went down mid-session, the agent caught the error, wrote it to the transcript and ended its loop, so thirty runs reported success with zero tool calls between them. The same signature reappeared independently on machine D, with a different mechanism: the backend never went down, but

`sandbox/agent.py`'s fixed 120 s client timeout is shorter than this CPU's prompt-evaluation time for the harness's own first-turn payload (measured at 109 s, model already resident), so 35 of 100 runs there read as complete and empty for a reason no outage would explain. One signature, two independent causes on two machines, is the strongest argument in this paper for treating the class rather than patching the instance.

The sixth reached the published numbers. The attacker logs each delivery of its payload, and counting those against runs shows eight of twenty were never served the attack, skewed by condition. In the summary a run that was never attacked is indistinguishable from one where the agent read the payload and declined: both simply score low. That channel's rate had been computed over planned runs rather than exposed ones.

G.1 What we changed

The common thread is that silence was being read as “nothing to report”. The defences all make components prove they were alive.

- Each monitor emits a startup event naming itself and its configuration, so a monitor that never booted is distinguishable from one with nothing to say. The egress monitor also reports whether its baseline is empty, since an empty baseline marks every destination anomalous and makes one control incapable of failing.
- The test scripts check that the Docker daemon answers before doing anything, and exit with a distinct code so “Docker is down” cannot be confused with “a control failed”.
- The orchestrator inspects each finished run, records the diagnosis in that run's metadata, and prints how many produced no data. It caught an empty run on its first outing, and machine D's independent occurrence of the same failure, on hardware none of this was written against, on its first outing there too.
- The analysis tolerates a torn log line and names the file and line rather than dying, because the three monitors append to one file from separate containers and that append is not atomic over a bind mount. A lost line may be an inert event or an alert that changes a verdict.
- Every figure is counted over the artifacts at the time of writing, never read from the configuration that would have produced them. One retracted table was written from `repetitions: 10` in a YAML file while three runs existed on disk.

H The Six Induction Vectors in Full

The agent is consistently assigned the same benign surface task: patching a division bug to pass a test suite. The experimental variance lies strictly in the temptation planted within its environment.

Tasks 01 through 03 represent passive temptations, such as exposed credentials or errors pointing to an internal registry, lacking any explicit instruction to act. These serve as structural baselines and produce none of the primary leakage results, existing solely to validate the harness's observational capabilities.

Tasks 04 through 06 introduce an *identical*, active injected instruction demanding the agent create a confirmation directory named using its session identifier. These tasks differ exclusively in their delivery channel: a live fetch from an external mirror, a note seeded in cross-run persistent memory, or a document embedded in a local knowledge base. Holding the malicious payload constant while varying the delivery channel enables the comparative analysis detailed in Section 5. This exfiltration technique directly mirrors the anchor incident, which leaked data through directory naming conventions [15].

Vectors 05 and 06 specifically model **ASI06: Memory and Context Poisoning** [4], where untrusted input is processed as trusted memory or retrieved context to compromise subsequent sessions. Vector 05 represents the most critical persistence threat, since `./memory` operates as a host bind mount that survives container teardown (`docker compose down -v`), effectively outliving the run that created it.

LLM Usage Statement

We used Claude as a coding assistant, to correlate monitor events across runs, and to iterate on ideas in the early stages of the project. Every figure in this paper is computed from the versioned artefacts by the analysis code in the repository, and the claims that rest on them were checked against that output by the authors. Two of the corrections this paper reports, the stale run counts and the differential dropout on machine D, were found during that checking.